



Bachelor's Degree in Computer Science, Networks, and Telecommunications Engineering

Final Course Project

Intermediate Report

2025-2026 Academic Year

EduCal – Academic Planning Application

Coordinators:

Filipe Freitas

Sandra Ferreira

Student:

Daniel Santos nº 51701

May 2026

Index

1. Introduction	2
2. Problem Description and Objectives	3
2.1. Problem Description	3
2.2. Objectives.....	3
3. System Requirements.....	3
3.1. Functional Requirements	3
3.2. Non-Functional Requirements.....	4
4. System Architecture	5
4.1. Data Model and Entity-Relationship Diagram.....	6
4.1.1. Core Scheduling Entities.....	7
4.1.2. Classification and Management	7
4.1.3. Automation and Business Rules	7
4.2. Security and Auditing.....	7
5. Implementation Status	8
5.1. User Authentication and System Access (RF06)	8
5.1.1. Authentication Interface	8
5.1.2. User Management Dashboard	9
5.1.3. Security Implementation.....	9
5.2. Interactive Academic Calendar and Event Logic (RF01)	10
5.2.1. Comprehensive Calendar Views.....	10
5.2.2. Advanced Event Management.....	11
5.2.3. Enhanced UX/UI Features	11
5.3. Technical Architecture Progress	11
5.3.1. Modular Backend Logic (Controller-Service Pattern).....	12
5.3.2. Performance and Session Management.....	12
5.3.3. Structural Integrity and Validation	12
6. Schedule.....	13
7. Next Steps.....	14
8. Intermediate Conclusion.....	15
References	15

1. Introduction

As part of the Bachelor's Degree in Computer Science, Networks, and Telecommunications Engineering, this report details the development of the final course project entitled EduCal – Academic Planning Application. The project, included in the 2025-2026 academic year, arises from the need to optimize the management of services involving critical planning, execution, and improvement activities.

Currently, these activities take place on dates set annually. However, there is no tool to facilitate the transition from one calendar year to the next, respecting specific criteria and deadlines. The current scenario for planning annual tasks is based on dense and complex spreadsheets (Excel). Although detailed, these tools are prone to filling errors and require an exhaustive manual effort to replicate the network of objectives, requesters, and instructions in the new school year.

The EduCal project aims to replace this manual process with an intelligent web-based application system. Unlike general-purpose calendars, which are excellent for personal scheduling but lack specific institutional logic, EduCal is designed specifically for academic planning through several key differentiators. Primarily, it utilizes hierarchical event logic where an "Event" represents a high-level academic objective linked to multiple independent "Occurrences," allowing for complex metadata, such as institutional classifications and assigned responsible personnel, to be linked to each entry. Furthermore, the system's primary value proposition is the "Migration Engine", which does not simply copy events but calculates the next year's calendar by automatically adjusting dates based on rules, such as shifting dates to avoid weekends or national holidays. Additionally, EduCal includes institutional workflow metadata, such as planning status and responsibility tracking, and provides formal institutional output through formatted PDF schedule exports for official distribution.

Given the nature of the project, its initial implementation is expected to take place within ISEL's services. To this end, the solution will be developed using modern technologies such as Next.JS [1] and PostgreSQL [2] and will be designed to be hosted on the institution's internal network. The current version of the prototype already implements a decoupled architecture that separates business logic from data storage, ensuring high maintainability.

The remainder of this report is organized as follows to provide a clear view of the project's evolution. Section 2 describes the problem and the project's core objectives. Section 3 details the functional and non-functional requirements. Section 4 presents the system architecture, including the refined Entity-Relationship data model. Section 5 reports the current implementation status, focusing on authentication, security, and the interactive calendar logic. Section 6 provides an updated project schedule reflecting the current progress. Section 7 outlines the upcoming milestones, such as the migration engine development. Finally, Section 8 summarizes the progress and findings in the intermediate conclusion.

2. Problem Description and Objectives

2.1. Problem Description

The current management of the academic calendar relies on dense and complex spreadsheets. Since there is no tool to automate the transition of dates between academic years, the process is manual, exhaustive, and prone to errors. In addition to dates, it is necessary to manually replicate a complex network of objectives and instructions, which increases the risk of incorrect scheduling.

2.2. Objectives

EduCal aims to replace manual effort with a web application. The main objectives are:

- **Baseline Creation:** Create an academic calendar from scratch as a base for the rest of the project and its functionalities.
- **Automation:** Automatically migrate the calendar from the current year to the next.
- **Application of Rules:** Ensure compliance with business rules, such as excluding holidays and weekends or events outside of school vacation periods.
- **Centralization:** Unify the workflow in a system that eliminates redundancies and ensures data integrity.

3. System Requirements

To ensure the EduCal application effectively addresses the identified problems and achieves its core objectives, a detailed set of requirements has been defined. These specifications serve as the technical blueprint for the development process, categorizing the system's needs into functional capabilities and non-functional quality standards.

3.1. Functional Requirements

Functional requirements outline the specific features, tasks, and behaviors the platform must provide to satisfy user needs. These define exactly what the system does, ranging from the fundamental management of academic events to the automated execution of the migration engine.

- **RF01 – Event and Occurrence Management:** The system must allow the creation and management of events that have multiple annual occurrences, each with its own descriptions and dates.

- **RF02 – Migration Engine:** It must be possible to transfer the event history from one academic year to the next, automatically validating the dates.
- **RF03 – Rule Configuration:** The system must allow flexible rules (e.g., working days) to be defined, which the migration engine must respect.
- **RF04 – Notification Templates:** The platform must allow the configuration of an email or other type of text templates associated with events for sending proactive notifications.
- **RF05 – Schedule Export (Optional):** The system must generate an academic calendar in PDF format for institutional distribution.
- **RF06 – Profile Access Control (Optional):** Two types of users have been defined to ensure data security:
 - **Viewer:** Has read-only access to consult planning and schedules.
 - **Editor:** Has full permissions to perform CRUD operations, configure rules, and run the migration engine.

3.2. Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and performance criteria of the system. Rather than focusing on specific behaviors, they define how the system performs, ensuring that the application is secure, maintainable, and provides good user experience.

- **RNF01 – Data Integrity:** The rules for “business days” and holidays or school terms must be inviolable during the migration process.
- **RNF02 – Usability:** The interface must be intuitive and capable of presenting the necessary density of information.
- **RNF03 – Architecture and Scalability:** The system must be modular and scalable, ensuring fast access to extensive calendars.

4. System Architecture

To ensure the project's viability and security, the architecture was designed based on robust technologies that are widely adopted in the industry. The choice of an infrastructure hosted on ISEL's network ensures that the academic calendar service is integrated into the institutional ecosystem.

- **Frontend and Backend (Next.JS):** Use of a single framework to manage both the interface and the business logic and API routes.
- **Database (PostgreSQL):** Choice of a relational system to ensure the normalization and security of calendar data.
- **Hosting:** It is recommended to use a virtual machine within the ISEL network to host both the HTTP service and the database.

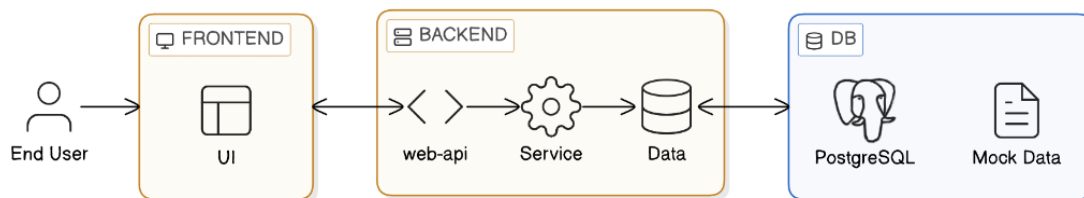


Figure 1 - System Architecture Diagram

As illustrated in Figure 1, the system follows a modern three-tier architecture to separate concerns and ensure maintainability:

- **Presentation Layer (Frontend):** Represented by the "UI" block in Figure 1, this layer built with Next.js, manages the state of complex calendar views (Daily to Yearly) and provides specialized interfaces for RF01, such as dynamic forms for multi-occurrence events.
- **Logic Layer (Backend):** The middle tier, shown in Figure 1 as the "web-api", "Service" and "Data" blocks, is divided into three key components to organize the code effectively:
 - **Web-API:** Acts as the entry point, managing secure communication, ensuring that only authenticated users with "Editor" or "Administrator" roles (RF06) can modify the institutional calendar and routing them to the correct logic.
 - **Service:** This is the most critical component of EduCal by being the bridge between our entry point and our data layer. It houses the Migration Engine (RF02) and the Business Rule Validator (RF03). It is responsible for the complex logic of calculating future dates, shifting occurrences to avoid weekends or holidays, ensuring all data adheres to the defined constraints.

- **Data:** A dedicated layer for data access, abstracting the communication between the logic and the storage system.
- **Data Layer (DB):** As shown in Figure 1, a PostgreSQL relational database provides robust and persistent storage for events, occurrences, and system configurations. The architecture also accounts for Mock Data during the development and testing phases to ensure the system is reliable before connecting to live institutional data.

Currently, all components outlined in the architecture diagram have been successfully implemented, except for the PostgreSQL data module. This is not a critical blocker due to the project's decoupled architecture; the business logic is independent of the specific storage technology.

4.1. Data Model and Entity-Relationship Diagram

To support the transition to an automated system, a relational database schema was designed, which is visually outlined in the Entity-Relationship Diagram presented in Figure 2. This figure serves as a generalized blueprint of the application's data layer, depicting various institutional components, such as users, academic events, and business rules.

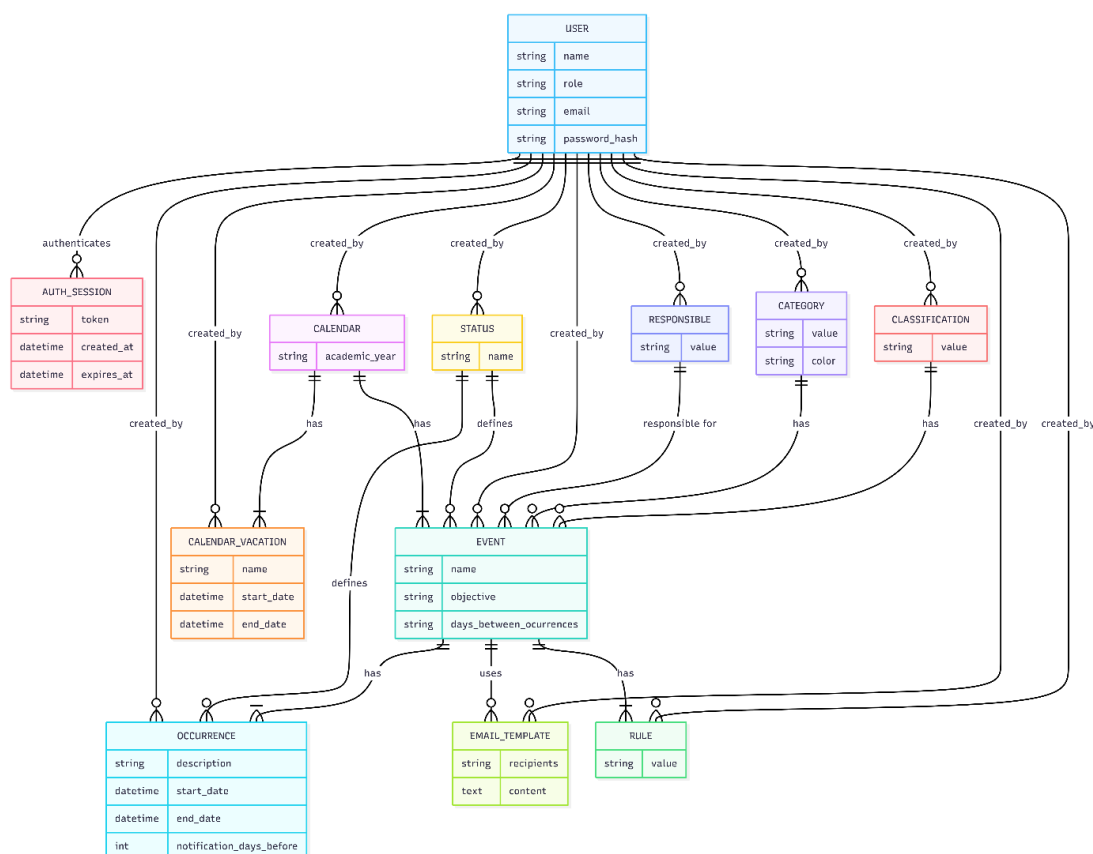


Figure 2 - Entity-Relationship Diagram

4.1.1. Core Scheduling Entities

The system's core logic centers on the relationship between Events and Occurrences:

- **Event:** Stores high-level metadata such as the objective and the frequency logic (*days_between_occurrences*).
- **Occurrence:** A concrete instance of an event with specific *start_date* and *end_date*. Each event can have multiple occurrences, allowing the system to track different sessions of the same activity.

4.1.2. Classification and Management

To facilitate filtering and institutional reporting, the system employs several normalization entities:

- **Status, Category, and Classification:** These entities allow users to categorize events (e.g., "Critical", "Administrative") and track their current state in the planning workflow.
- **Responsible:** Tracks the personnel or department accountable for specific events, ensuring clear institutional ownership.

4.1.3. Automation and Business Rules

The model includes specific entities to support the project's automation requirements:

- **Rule:** Stores the rules list used by the Migration Engine to shift dates during the academic year transition.
- **Email_Template:** Supports proactive notifications (RF04) by storing pre-configured content and recipient lists linked to specific academic events.
- **Calendar_Vacation:** Defines non-working periods that the system must respect when validating or migrating dates.

4.2. Security and Auditing

- **User and Auth_Session:** These entities manage the multi-role access control (RF06). The User entity stores cryptographic hashes of passwords, while Auth_Session tracks active security tokens and their expiration.

- **Auditing:** Nearly every entity in the model is linked to a User via a *created_by* relationship, ensuring a full audit trail of who modified the institutional calendar.

5. Implementation Status

Since the delivery of the initial report, the development of EduCal has progressed from the architectural planning phase to the functional implementation of its core modules. This section details the current state of the application, focusing on the realized functional requirements (RF01 and RF06) and the technical measures taken to ensure system robustness.

5.1. User Authentication and System Access (RF06)

The foundation for a secure and multi-role environment has been established. The system now includes complete authentication flow and a centralized user management interface.

5.1.1. Authentication Interface

Figure 3 shows the implemented login portal with email and password authentication to manage access to the platform. To enhance security and user experience, the system supports persistent sessions with a "Remember Me" functionality.



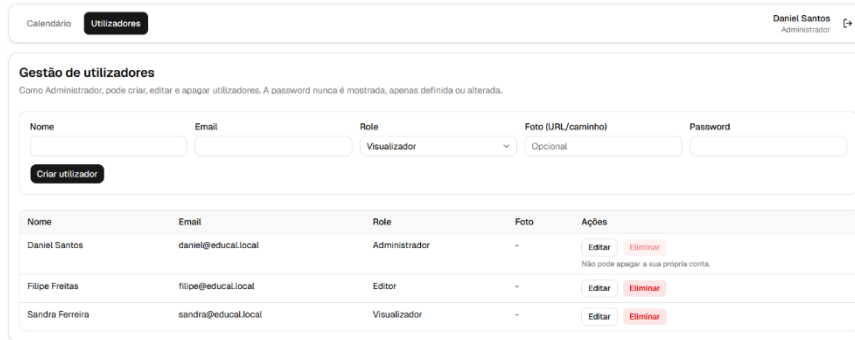
The screenshot shows a login form with the following elements:

- Email:** A text input field.
- Password:** A text input field with a toggle icon on the right.
- Remember Me:** A checkbox labeled "Manter sessão iniciada" (Keep session started).
- Login Button:** A black button labeled "Entrar" (Enter).

Figure 3 - Login Interface

5.1.2. User Management Dashboard

The administrative dashboard, displayed in Figure 4, allows Administrators to perform CRUD operations on user accounts, assigning specific roles: Administrator, Editor, or Viewer. This interface also allows us to visualize all the user data, except the password, for privacy reasons.



Calendário Utilizadores Daniel Santos Administrador

Gestão de utilizadores
Como Administrador, pode criar, editar e apagar utilizadores. A password nunca é mostrada, apenas definida ou alterada.

Nome Email Role Foto (URL/caminho) Password

Criar utilizador

Nome	Email	Role	Foto	Ações
Daniel Santos	daniel@educal.local	Administrador	-	Editar Eliminar Não pode apagar a sua própria conta.
Filipe Freitas	filipe@educal.local	Editor	-	Editar Eliminar
Sandra Ferreira	sandra@educal.local	Visualizador	-	Editar Eliminar

Figure 4 - User Management Dashboard

5.1.3. Security Implementation

All user passwords are encrypted using modern hashing algorithms [4] before being stored in the data module. Also, security is further reinforced by storing session tokens in Secure and HTTP Only cookies [3], preventing client-side script access (XSS protection) and configured with expiration intervals to mitigate the risk of unauthorized access through session hijacking.

5.2. Interactive Academic Calendar and Event Logic (RF01)

The primary functionality of the application, the interactive calendar, is now operational, replacing the previous dependency on complex spreadsheets.

5.2.1. Comprehensive Calendar Views

The interactive calendar shown in Figure 5 represents the core of the EduCal application, to ensure high information density and usability, providing multiple temporal perspectives: **Daily**, **Weekly**, **Monthly**, and **Yearly** views. Also allows filters, creation of new events and a settings toggle for some UI/UX customization.

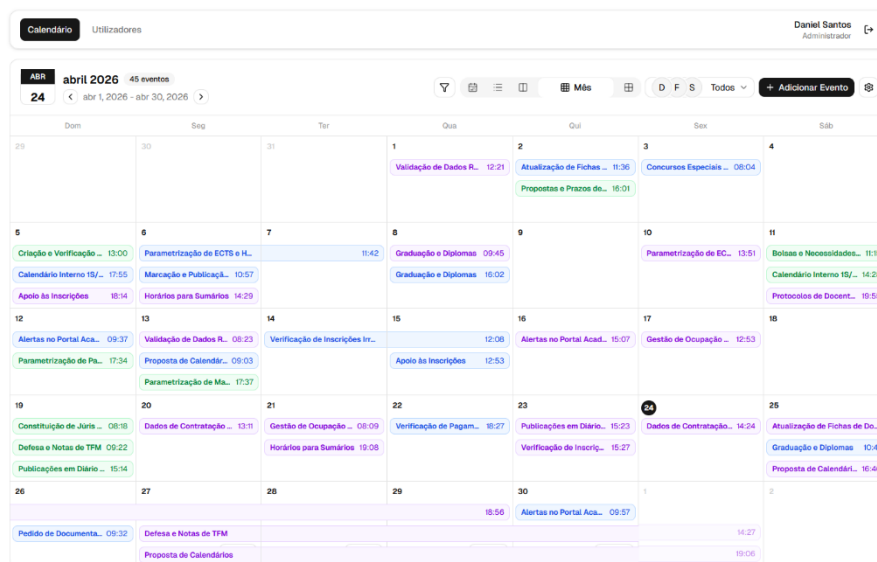


Figure 5 - Main Calendar Interface (Month View)

5.2.2. Advanced Event Management

Figure 6 illustrates the "Add New Event" dialog, by allowing users to define:

- **General Event Metadata:** Name, objective, category, classification, and responsible personnel.
- **Multiple Occurrences:** A single event can trigger multiple instances, each with specific descriptions and start/end dates.

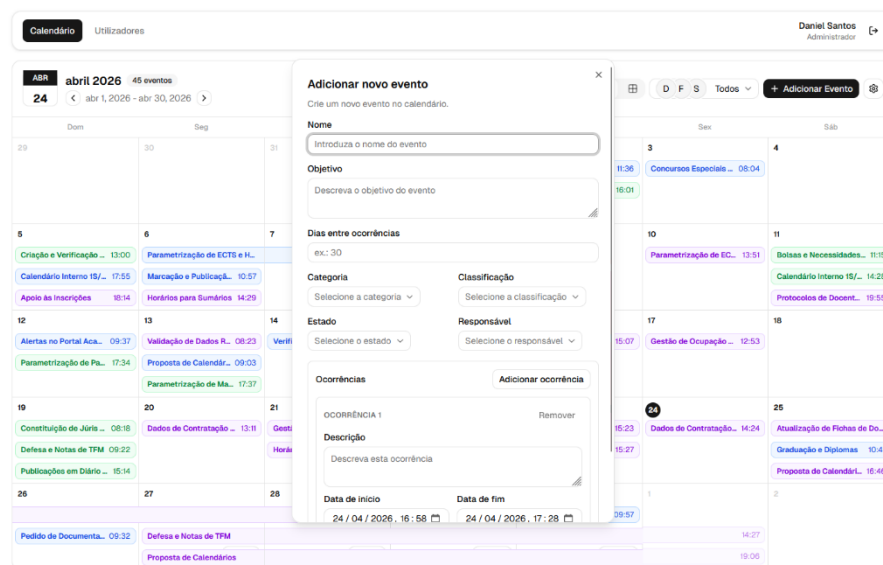


Figure 6 - Add event dialog

5.2.3. Enhanced UX/UI Features

Moving beyond static data entry, the calendar supports **drag-and-drop** interactions, allowing for intuitive rescheduling of occurrences directly on the grid. This fluidity is critical for meeting the project's goal of centralizing and optimizing the workflow.

5.3. Technical Architecture Progress

The system continues to adhere to the planned three-tier architecture, with significant progress made in the robustness of the backend services. The current source code and project documentation are managed in a GitHub repository¹. The current implementation leverages the Next.js App Router not just for rendering, but as an API layer with capabilities for easy scalability.

¹ The source code is available at: <https://github.com/Patilhas/educal>

5.3.1. Modular Backend Logic (Controller-Service Pattern)

To ensure high maintainability and follow clean code principles, the backend is structured using a Controller-Service-Repository pattern:

- **Controllers:** Entities like the *authController* and *calendarController* act as orchestrators, handling the incoming HTTP requests and managing the response lifecycle.
- **Services:** All business logic, such as user validation and event occurrence calculations, is encapsulated in specialized services (e.g., *authService*). This ensures the logic is decoupled from the API routes and is easily testable.
- **Data Abstraction:** The code utilizes a dedicated data access layer that currently handles Mock Data. This was implemented through a repository interface, allowing the system to be "PostgreSQL-ready". Switching to the final database will only require changing the data provider without modifying the business services.

5.3.2. Performance and Session Management

The backend implementation introduces advanced technologies to ensure the application is production-ready:

- **Stateful Session Management:** The system has the capability to store active sessions, providing a low-latency environment, ensuring that user permissions are verified in real-time without overloading the main database.
- **Authentication Middleware:** Access control is enforced through custom higher-order functions (*withApiAuth* and *withApiAuthContext*). This middleware intercepts requests to protected routes, validating the Secure and HTTP Only session cookies before allowing the request to proceed to the controllers.

5.3.3. Structural Integrity and Validation

To prevent data corruption issues and easy payload validation, the backend enforces:

- **Schema Validation (Zod):** The application uses the Zod library to define and enforce data schemas for every API input. Whether creating a new user or a multi-occurrence event, the backend ensures data conforms to the required types and constraints before it is processed or stored.

6. Schedule

The updated project timeline is presented in the Gantt Chart (Figure 7). This visual representation tracks the progress of the three primary tracks: Deliverables, Backend, and Frontend, supplemented by two continuous activities that span the entire project lifecycle. There are some tasks that are even going better than the timeline established in the initial report:

- **Continuous Activities:**

- **Reporting:** Documentation and report writing are treated as ongoing tasks from the project's start on March 13th through to the final submission.
- **Testing:** A dedicated testing phase runs parallel to all development activities to ensure system stability and requirement compliance throughout the build.

- **Deliverables and Milestones:**

- **Initial Report Delivery:** Completed on March 13th, 2026.
- **Intermediate Report Delivery:** Set for May 1st to assess progress.
- **Final Report Delivery:** The project concludes with the final submission on July 11th.

- **Backend Development:**

- **API Architecture and Setup:** Successfully concluded in mid-April, establishing the core logic layer.
- **Business Logic and Database:** Almost finished, missing the migration to the PostgreSQL database.

- **Frontend Development:**

- **Base Interface:** The UI design and initial prototyping phases are finished.
- **Feature Development:** The implementation of the primary calendar engine and user management is ahead of schedule, allowing for an earlier focus on system security and UX refinement. Next step on this task will be the implementation of the rest of the functional requirements.

- **Polishing:** A final refinement phase is scheduled for the last week of June to ensure a professional finish before the final delivery

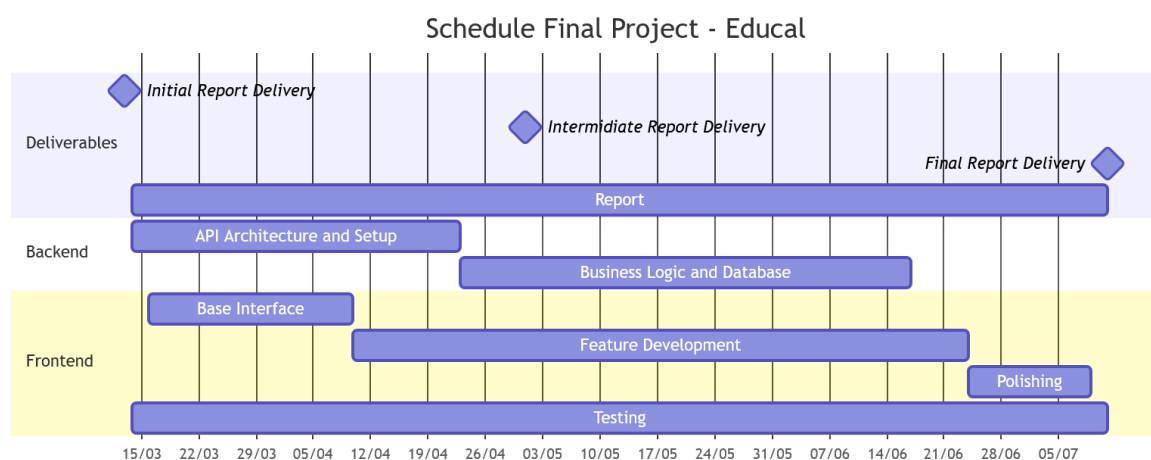


Figure 7 - Schedule

7. Next Steps

The second half of the project will focus on the transition from a functional interface to an automated institutional tool. The upcoming tasks are:

1. **PostgreSQL Integration:** Replacing the current Mock Data module with the persistent relational database layer.
2. **Migration Engine (RF02):** Implementing the core logic to automate the transition of event occurrences from the 2025/2026 calendar to the 2026/2027 year and the following years.
3. **Business Rule Automation (RF03):** Programming the engine to respect holidays, weekends, and academic-specific constraints during the migration process.
4. **Notification System & Templates (RF04):** Developing the email notification engine for event stakeholders.
5. **Schedule Export (RF05):** Implementing the PDF export functionality for institutional distribution.
6. **Polishing:** Finalizing the UI/UX and making improvements according to future users feedback.

8. Intermediate Conclusion

At this intermediate stage of the project's timeline, the EduCal project has successfully moved from conceptual requirements to a robust, functional prototype. The core challenges regarding user management (RF06) and complex event logic (RF01) have been addressed, resulting in an intuitive and secure web-based application that already demonstrates significant improvements over the legacy spreadsheet system.

The decision to prioritize decoupled architecture has proven effective, allowing for rapid UI/UX development while maintaining a clear path for database integration. With the foundational calendar system operational, the project is well-positioned to tackle the development of the Migration Engine. It is all according to schedule to deliver a complete, high-quality solution by the final deadline on July 11th.

References

- [1] **Next.JS** (2026) - *Next.js Documentation* (<https://nextjs.org/docs>)
- [2] **The PostgreSQL Global Development Group** (2026) - *PostgreSQL Documentation* (<https://www.postgresql.org/docs/>)
- [3] **OWASP Foundation** (2026) - *Session Management Cheat Sheet & XSS Prevention* (<https://cheatsheetseries.owasp.org/>)
- [4] **Node.js Foundation** (2026) - *Crypto Module and Hashing Standards* (<https://nodejs.org/api/crypto.html>)

Report delivered on 1st May 2026